Executive Summary

This report presents the drift detection and mitigation pipeline developed by Team Green Beans for the NAISC 2026 challenge, hosted by Singtel. The challenge requires participants to build a fully automated system that can identify and correct distributional shifts between training and test data, then produce churn probability predictions on a hidden dataset of up to 10 million rows and 500 features.

Our pipeline operates in five sequential stages:

1. Loading and preparing the data
2. Classifying every column by its statistical type, and detecting drift using tailored statistical tests
3. Ranking drifted features by urgency
4. Applying targeted fixes to drifted columns
5. Training a LightGBM classifier to produce churn probability predictions

Every stage is designed to be column-agnostic, meaning the pipeline adapts to whatever data it receives at runtime without relying on assumptions about specific column names or values.

The approach improved prediction performance from a baseline Area Under the Precision-Recall Curve (AU-PRC) of **0.5082** to a mitigated score of **0.7774**, a gain of **0.2692** points. A key finding during development was the discovery that a feature flagged as severely drifted with a PSI score of **2.72** was in fact a data quality issue caused by inconsistent text formatting, which the pipeline resolved automatically, reducing the PSI to **0.006**. This finding shaped several core design decisions described in the sections that follow.

Chapter 1: Introduction to the Competition

1.1 Introduction

The National AI Student Challenge (NAISC) 2026, hosted by Singtel, tasks participating teams with building an automated machine learning pipeline that detects and corrects data drift before making churn predictions on an unseen customer dataset. Data drift occurs when the statistical properties of data change over time, causing a model trained on historical data to perform poorly when deployed on newer data. The core difficulty of the challenge is not building a model that performs well on familiar data, but building a system that remains reliable when the data it encounters at prediction time looks different from the data it was trained on.

The public dataset provided consists of 70,430 training rows from January to October 2025, and 14,086 test rows from November and December 2025. Each row represents one customer, with 45 columns describing their demographics, service usage, billing behaviour, and contract details. The target variable is ChurnStatus, which records whether a customer churned within the observation period. The churn rate is stable across both splits at approximately 28.8%, confirming that the challenge is one of covariate drift, where the input feature distributions shift over time, rather than a change in the underlying relationship between features and churn.

Three constraints shape the design of any viable solution. First, the pipeline must generalise to a hidden dataset containing up to 10 million rows and up to 500 features, none of which are known in advance. Column names, data types, and category values may all differ from the public dataset, meaning any solution that hardcodes assumptions about specific columns

will fail on the hidden data. Second, the model hyperparameters are fixed by the competition and identical for all teams, ensuring that differences in performance between submissions reflect the quality of drift handling rather than model tuning. Third, the entire pipeline must complete within 10 minutes on a CPU-only machine. To meet this constraint at scale, the pipeline is built on Polars, a high-performance data processing library that computes all column statistics in a single parallel pass rather than column by column. Distribution statistics are computed on a sample of up to 50,000 rows for speed, while cardinality checks always run on the full dataset to ensure accuracy. This design keeps Stage 2 running in under two seconds on the public dataset, with estimated headroom well within the time limit even at 10 million rows.

These three constraints collectively define the design philosophy of our pipeline: every decision must be justifiable on the basis of what the data shows at runtime, must apply equally to any tabular dataset with a time column and a binary target, and must be efficient enough to complete within the time limit at scale.

1.2 Libraries and Dependencies

The pipeline draws on two categories of libraries: Python's standard library for basic input/output and system operations, and a set of third-party packages for data processing, statistical testing, and machine learning. The choice of each third-party library was deliberate and is explained below.

1.2.1 Third-party Libraries

Polars is the primary data processing library used throughout Stages 1 to 4. It was chosen over the more commonly used pandas library for two reasons directly relevant to the competition constraints. First, Polars executes aggregations in parallel across all available CPU cores, whereas pandas processes columns sequentially. On a dataset with up to 500 features, this difference is significant: computing statistics for every column in a single parallel pass takes a fraction of the time that sequential processing would require. Second, Polars uses the Apache Arrow memory format, which avoids copying data between operations and keeps memory usage low even at 10 million rows. The pipeline uses pandas only once, as a thin handoff layer immediately before model training, because LightGBM requires a pandas DataFrame as input. Every operation before that point is Polars-native.

Pandas is mainly used for basic input/output operations. As LightGBM requires pandas DataFrame as input, it serves as a handoff layer between Polars and LightGBM in Stage 5. It is also used in the dashboard feed export to construct and write to CSV. The drift analysis results are in the form of Python dictionaries and lists at that stage, thus making pandas a convenient choice for this operation.

PyArrow is a required runtime dependency for Polars to internally handle memory layout conversion between its own columnar format and pandas format. This is crucial in Stage 5 when using .to_pandas() for the conversion before passing into LightGBM. PyArrow therefore facilitates zero-copy memory conversion between the two formats and without it, these conversions would fail at runtime.

LightGBM is the gradient boosting classifier used in Stage 5. It is specified by the competition and its hyperparameters are fixed for all teams. LightGBM is well-suited to this task because it handles mixed feature types natively, treats missing values internally without requiring imputation, and trains efficiently on large tabular datasets on CPU. The competition's use of LightGBM as the fixed model means that all performance differences between teams are attributable to drift handling rather than model selection.

scikit-learn is used exclusively for computing the AU-PRC metric via its average_precision_score function, and the confusion matrix via the confusion_matrix function. No scikit-learn preprocessing, encoding, or modelling components are used. These functions were chosen because they are standard implementations in the Python ecosystem and their outputs are consistent with how the competition metrics are defined.

SciPy provides the statistical tests used in Stage 2: the two-sample Kolmogorov-Smirnov test for numerical columns, the chi-squared goodness-of-fit test for categorical columns, and the normal cumulative distribution function used to compute p-values for the two-proportion Z-test on sparse columns. SciPy was chosen because it provides peer-reviewed, well-tested implementations of these tests that match the mathematical definitions used in academic and industry drift detection literature.

NumPy is used for numerical operations within the detector and classifier modules: computing safe float conversions, handling not-a-number values, and constructing the probability arrays required by the Jensen-Shannon divergence calculation. It also provides the random number generator used for reproducible sampling in Phase B tests. NumPy is the standard numerical computing library in Python and underpins both SciPy and Polars internally.

joblib is used in Stage 5 to serialise the trained LightGBM model to a file called model.joblib. This file is a required submission artefact. joblib is the standard serialisation library for scikit-learn compatible models and produces a format the judges can load and inspect directly.

Streamlit is used to serve the interactive drift dashboard as a local web application after the pipeline run completes. It was chosen because it allows fast, lightweight deployment of data-app interfaces directly from Python with minimal frontend overhead.

Plotly is used for all interactive dashboard visualisations, including drift distributions, mitigation comparisons, confusion matrix rendering, and metric charts. It was chosen because it provides high-quality interactive plotting with precise control over layout, annotation, and styling needed for the dashboard narrative.

1.2.2 Standard Library Modules

The following modules are part of Python's standard library and require no installation.

io and **sys** are used together at the start of the pipeline to reconfigure the standard output and error streams to use UTF-8 encoding. This prevents the pipeline from crashing on Windows terminals that default to a different encoding when the output contains characters outside the basic ASCII range.

argparse parses the two command-line arguments the pipeline accepts: the path to the training file and the path to the test file. It validates that both arguments are provided and generates a usage message if the pipeline is called incorrectly.

pathlib provides the Path class, which is used throughout the pipeline to construct file paths in a way that works correctly on both Windows and Unix systems. The output files, prediction.csv and model.joblib, are written to the project root using pathlib path construction.

time is used to record the wall-clock start time at the beginning of the pipeline and compute the total elapsed runtime printed in the summary table at the end.

dataclasses and **enum** are used in report.py and classifier.py to define the typed data structures that carry information between stages: the FeatureType and DriftSeverity enumerations, and the ColumnProfile, ColumnDriftResult, and DriftSummary dataclasses. Using typed structures rather than plain dictionaries makes the contracts between stages explicit and reduces the risk of silent errors when fields are missing or misnamed.

typing provides type hint annotations used throughout the codebase for Dict, List, Optional, and Tuple. These annotations have no effect at runtime but make the expected inputs and outputs of each function explicit, which aids readability and reduces integration errors.

warnings is used in the detector to suppress SciPy warnings that would otherwise print to the console during chi-squared tests on small expected cell counts. The suppression is scoped to the specific operation rather than applied globally.

Chapter 2: Detection, Mitigation, and Validation

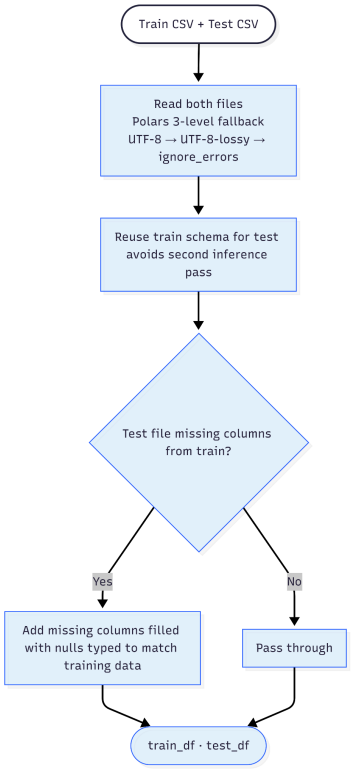
2.1 Detection and Mitigation Strategy

When a machine learning model is trained on historical data and then deployed to make predictions on new data, it assumes the world has not changed too much in between. In practice, this assumption rarely holds. Customer behaviour shifts, business processes evolve, and even the way data is recorded can change from one month to the next. This is known as data drift, and if left unaddressed, it causes a model to make predictions based on patterns that no longer reflect reality.

The challenge is that not every statistical difference between two datasets is meaningful. A large shift in a number can reflect a genuine change in behaviour, a seasonal pattern that was always present, or simply an inconsistency in how the data was recorded. A detection system that treats all three the same way will make poor decisions about what to fix, and may end up modifying features that were working correctly.

Our pipeline addresses this through five sequential stages, each with a clearly defined responsibility.

2.1.1 Stage 1: Load Data



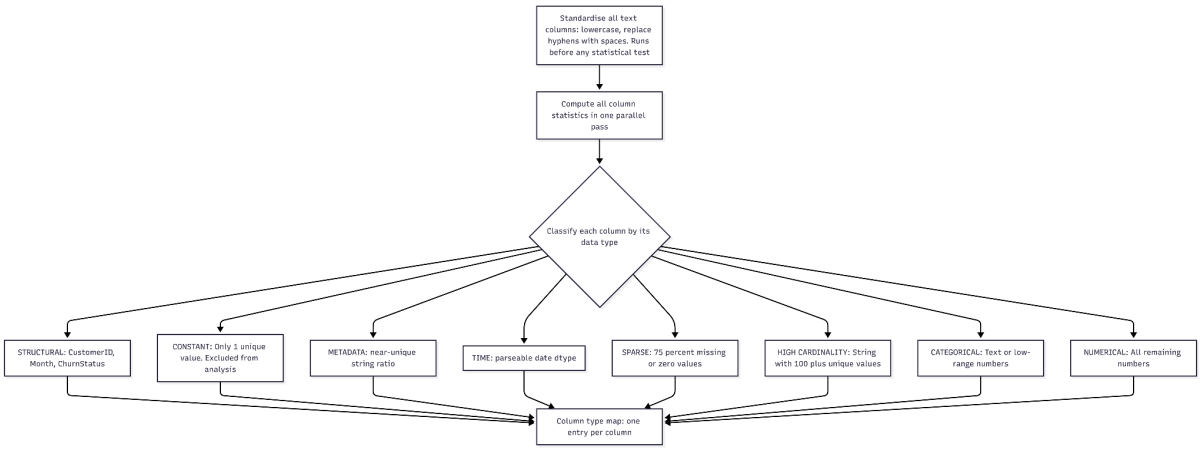
Before any analysis begins, the pipeline loads the training and test files and prepares them for processing. This step handles three practical problems that commonly arise with real-world data files.

The first is encoding inconsistency. Files from different systems are sometimes saved in different text encodings, which can cause a script to crash or misread values on the first attempt. The pipeline tries three reading strategies in sequence, falling back to a more permissive approach each time, so that files are loaded correctly regardless of how they were exported.

The second is missing value representation. In raw data files, a missing value might be recorded as a blank cell, the word "null", "NA", "NaN", or simply nothing at all. The pipeline treats all of these as equivalent missing values from the moment the file is read, preventing them from being misidentified as meaningful text later.

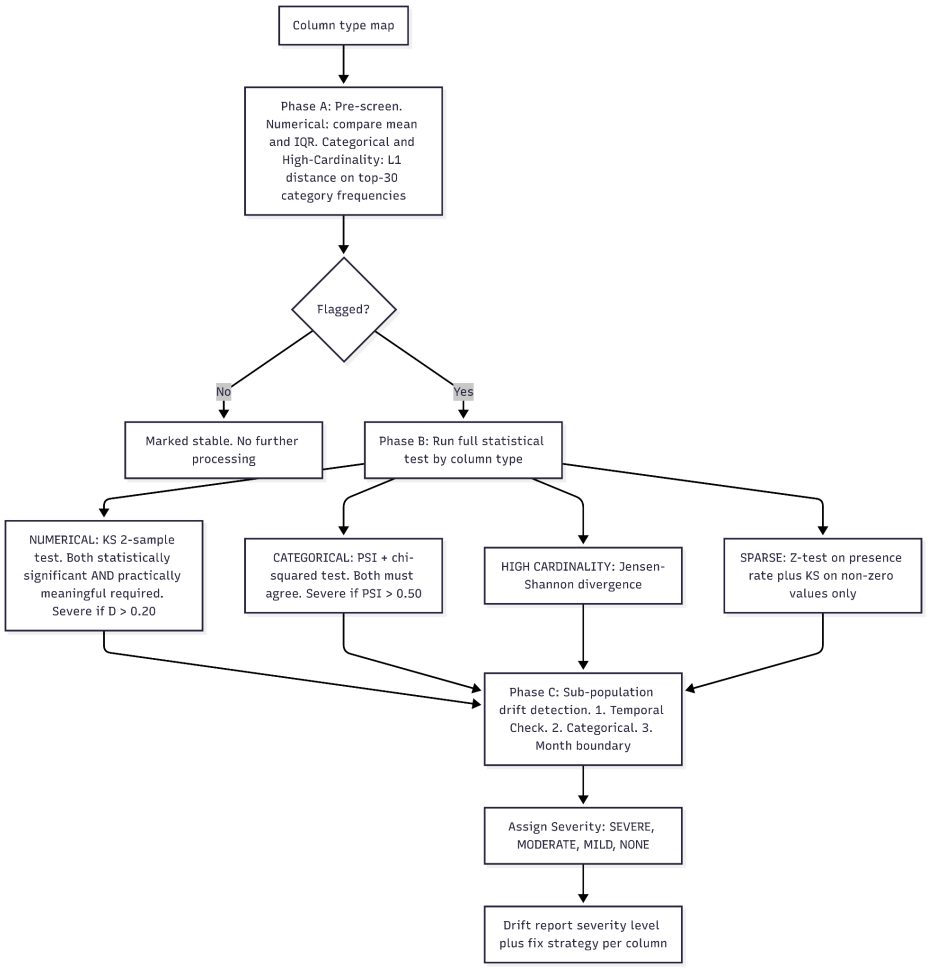
The third is column mismatch. If the test file is missing a column that exists in the training data, the pipeline fills that column with null values typed to match the training data rather than stopping with an error. This is a deliberate robustness measure for the hidden dataset, where the column structure is not known in advance.

2.1.2 Stage 2: Classify Columns and Detect Drift



Stage 2 is the analytical core of the pipeline. It has two responsibilities: first, classify every column by the kind of data it contains; second, test each column for drift using the statistical method appropriate to its type.

The classification step exists because different kinds of data require different tests. Applying the same test to a column of monthly charges and a column of contract type would produce unreliable results. The pipeline automatically sorts every column into one of seven categories based on its statistical properties, from identifier columns that are excluded from analysis entirely, to sparse columns with a high proportion of missing values, to standard numerical and categorical columns. This classification drives every downstream decision.



Once columns are classified, drift detection runs in three phases. The first phase is a fast pre-screen that uses only the summary statistics already computed during classification. For numerical columns, this includes the mean, standard deviation, and interquartile range. Categorical and high-cardinality columns first pass through a lightweight frequency-based pre-screen. Only columns whose category distributions shift meaningfully are sent to the deeper statistical test in Phase B. Comparing these summary statistics between training and test data requires no additional passes over the raw data and runs in milliseconds regardless of dataset size. Any column whose summary statistics fall within an acceptable range is marked stable immediately and does not proceed further. Only columns that show a meaningful shift at the summary level are passed to the second phase, which runs a full statistical test on each flagged column using the method appropriate to its type.

Column type	**Phase A: pre-screen**	**Phase B: deep test**	**Phase C: sub-population**	**What a high score means**
Numerical	Normalised mean shift + IQR overlap gap	KS 2-sample test	Temporal month-by-mont h profile comparison	The range and shape of values has shifted
Categorical	L1 distance on top-30 category frequencies	PSI + chi-squared test	Per-category proportion shift across values (categorical	The proportion of customers in each category has changed

			only)	
High cardinality	L1 distance on top-30 category frequencies	Jensen-Shannon divergence	Temporal-only check; no categorical sub-population upgrade	Frequency distribution across many unique values has shifted
Sparse	Absolute shift in presence rate	Z-test + conditional KS on non-zero values	Temporal-only check; no categorical sub-population upgrade	Presence rate has changed, or non-zero values have shifted

For numerical columns, drift is only confirmed in Phase B when two conditions are met simultaneously: the result must be statistically significant, and the effect must be large enough to be practically meaningful. This dual requirement prevents the pipeline from flagging trivial differences simply because the dataset is large enough to make any difference statistically significant. For categorical columns, both PSI and chi-squared must agree before drift is confirmed, reducing the risk of a false positive caused by the limitations of either test alone.

Following the deep statistical tests, the pipeline runs a third layer of analysis called Phase C: sub-population drift detection. Phase C runs for every feature that reached Phase B, regardless of whether drift was confirmed. It has three sub-tests.

The first is a temporal check: it computes the feature's per-month statistics across all training months and compares the most recent training months against the test months. If the recent training months are closer to the test distribution than the full training average, the drift is flagged as temporally concentrated, meaning it emerged late in the training period rather than being present throughout.

The second is a categorical sub-population check: for categorical features, it compares the proportions of each category value between training and test sets. If any individual category shows a proportion shift of more than 10%, sub-population drift is recorded. Critically, Phase C can upgrade features that passed Phase B but show sub-population drift: such features are reclassified as mildly drifted and receive mitigation despite passing the full-dataset test. This catches cases where drift in one customer segment is masked by an offsetting shift in another, which a whole-dataset comparison would miss entirely.

The third is a month-boundary check: for features where Phase C detects temporally concentrated drift, the pipeline records which calendar months account for the largest share of the shift. This is a reporting step only and does not affect severity; it provides an audit trail identifying whether the observed drift is tied to specific periods, such as a seasonal peak or a promotional event.

Each confirmed case of drift is assigned a severity level of mild, moderate, or severe, based on the size of the shift. These levels feed directly into Stages 3 and 4.

Applying this process to the public dataset identified eight features with confirmed drift, summarised below. TransactionMode, which initially showed a PSI of 2.72, was resolved by string normalisation during Stage 2 before formal drift testing and therefore does not appear as a confirmed drift feature. Its story is told separately below.

Feature	Type	Test	Severity	Fix applied
DigitalInvoicing	Categorical	PSI	Severe	Target encoding (Laplace m=10)
AvgMonthlyLongDistanceCharges	Numerical	KS	Severe	Quantile binning (10 deciles)
NumberofReferrals	Categorical	PSI	Mild	Target encoding (Laplace m=10)
ReferredaFriend	Categorical	PSI	Mild	Target encoding (Laplace m=10)
MonthlyCharge	Numerical	KS	Moderate	Quantile binning (10 deciles)
TotalLongDistanceCharges	Numerical	KS	Mild	Quantile binning (10 deciles)
PrioritySupport	Categorical	PSI	Mild	Target encoding (Laplace m=10)
Contract	Categorical	PSI	Mild	Target encoding (Laplace m=10)

The value of running string normalisation before statistical testing is best illustrated by what happened with a feature called TransactionMode, which records how a customer pays their bill. Unlike the eight features in the table above, TransactionMode was resolved during Stage 2 preprocessing and therefore did not proceed to formal drift testing.

On the first pass, the pipeline flagged TransactionMode as severely drifted, with a PSI score of **2.72**. To put this in context, a PSI score above **0.50** is considered severe; **2.72** is exceptionally high, and would ordinarily indicate that the distribution of payment methods had changed dramatically between training and test.

Before any statistical test ran, the pipeline applied its standard text standardisation step inside Stage 2, converting all category labels to lowercase and replacing hyphens with spaces. The PSI score fell immediately from **2.72** to **0.006**, well within the stable range.

The cause was straightforward. In the training data, the same payment method had been recorded two different ways: "Bank Withdrawal" and "bank-withdrawal". To the model, these appeared to be two separate categories. In the test data, this inconsistency had been corrected, leaving only one version. The apparent distribution shift was not a change in customer behaviour. It was a formatting inconsistency in the training data that had since been resolved.

Category label in training data	Category label in test data	PSI before normalisation	PSI after normalisation
Bank Withdrawal (35.0%)	Bank Withdrawal (53.0%)	2.72	0.006
bank-withdrawal (21.5%)	absent		
Credit Card (41.9%)	Credit Card (45.7%)		

A pipeline that had applied the statistical test without first standardising the text would have flagged TransactionMode as severely drifted and applied target encoding, replacing a categorical signal with churn rate estimates calculated from a corrupted category structure.

The correct order is to standardise first, then test. Because the pipeline standardises category labels during Stage 2 before formal drift testing, TransactionMode does not appear in the confirmed drift table above.

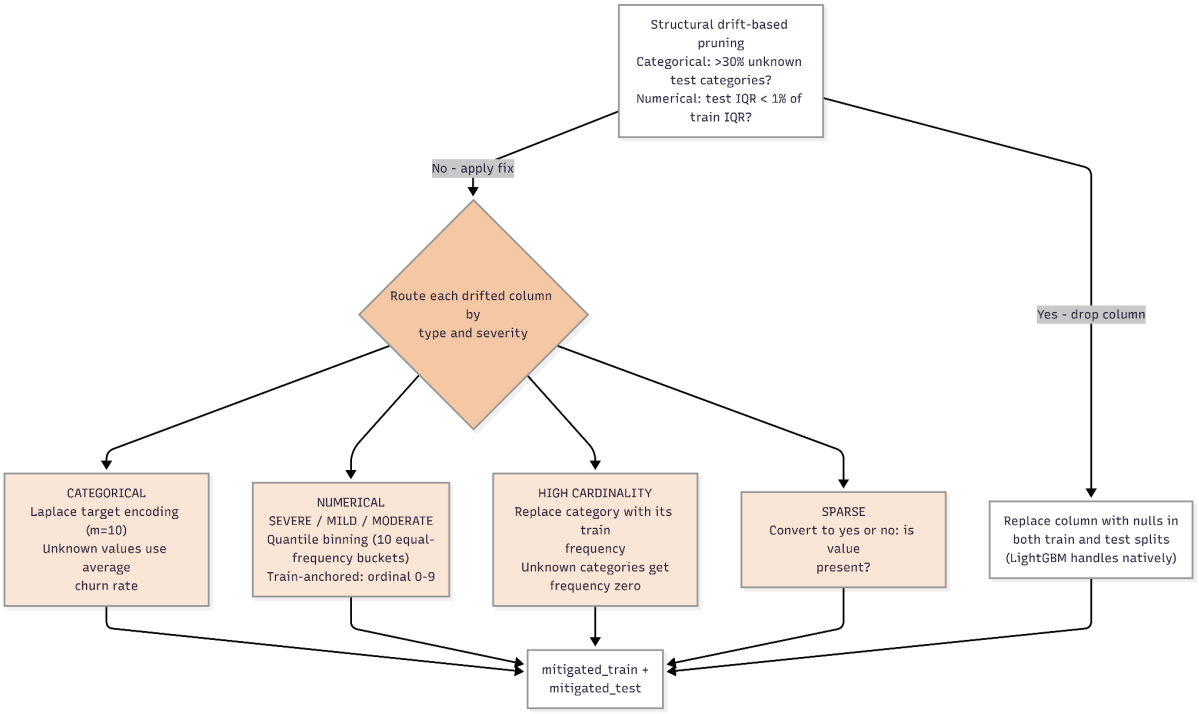
2.1.3 Stage 3: Prioritise by Recency

Stage 3 first computes month-level decay weights using **exp(lambda * rank).** It then computes weighted month fractions and sets **recency_factor = sum(w_fraction^2)**. Final ranking uses **weighted_score = raw_score × severity_weight × recency_factor.**

Because recency_factor is a single scalar applied to all drifted features, it rescales urgency globally and does not by itself change relative ordering between features. Instead, the pipeline computes a single recency concentration score from the test set's month distribution. Each month in the test set is assigned a rank from oldest to newest. The recency factor is computed as a concentration measure over the decay-weighted month distribution in the test set. When test rows are concentrated in recent months, the factor is larger; when they are spread more evenly across months, the factor is smaller. This single scalar is multiplied uniformly into every column's weighted score, meaning all drifted columns are adjusted by the same recency factor rather than each receiving an individually calculated weight.

Severity is weighted numerically: a severe classification contributes three times as much as a mild one, and a moderate classification contributes twice as much. A column that is both severely drifted and present in a test set concentrated in recent months will rank at the top of the list and be addressed first. This ranking is particularly useful when scaling to datasets with hundreds of features, where a clear priority order supports downstream review and mitigation planning.

2.1.4 Stage 4: Fix Drift



Stage 4 applies a targeted fix to each drifted column, chosen according to its type and severity. Although the detector records an intermediate mitigation tag during drift assessment, the final mitigation applied in Stage 4 is determined by the dedicated mitigation module. In the current pipeline, low-cardinality categorical drift is ultimately handled using target encoding. A single universal fix would be insufficient because the same correction that works for a drifted categorical column could be harmful when applied to a drifted numerical one.

Every fix in this stage is calculated using training data only. The parameters are then applied equally to both the training and test data. This is a strict requirement: if any information from the test data were used to calculate the fix, it would leak knowledge of the future into the model's training process, producing results that would not generalise to genuinely unseen data.

The fixes applied are:

Structural drift-based pruning is applied as a safety net before any other mitigation runs. Two conditions can trigger a feature to be dropped entirely. The first applies to categorical features: if more than 30% of test rows carry a category value that never appeared in training, the model has no basis for prediction on those rows, and any encoding would be unreliable. The second applies to numerical features: if the test interquartile range has collapsed to less than 1% of the training interquartile range, the feature has lost all distributional variation on the test side and the model's learned splits are no longer meaningful. In either case, the column is replaced with null values in both splits, which LightGBM handles natively. Neither condition fired on the public dataset, but the mechanism is in place to prevent silent prediction failures on the hidden dataset, where feature distributions are unknown

It is worth noting that string normalisation runs at two separate points in the pipeline. The first pass happens inside Stage 2 before any drift test runs, as part of preparing category value counts. This is what resolved the TransactionMode artefact described above. The second pass happens after all targeted fixes are applied, as the first step of Stage 5 immediately before the data is handed to the model. All text is converted to lowercase, stripped of leading and trailing spaces, and punctuation is normalised. This step runs automatically regardless of whether a column was flagged for drift, catching formatting inconsistencies like the TransactionMode case that appear in columns the statistical tests might otherwise have passed.

Categorical columns receive target encoding with Laplace smoothing. Each category label is replaced with a smoothed estimate of the churn rate for customers carrying that label, calculated from the training data. A smoothing factor of 10 is applied, which pulls the estimate for rare categories back towards the overall average churn rate, preventing the model from over-fitting to categories with very few examples.

smoothed rate = (n x category churn rate + 10 x global churn rate) / (n + 10)

Here n is the number of training customers in that category. When n is small, the formula pulls the estimate strongly towards the global average. When n is large, the observed churn rate dominates.

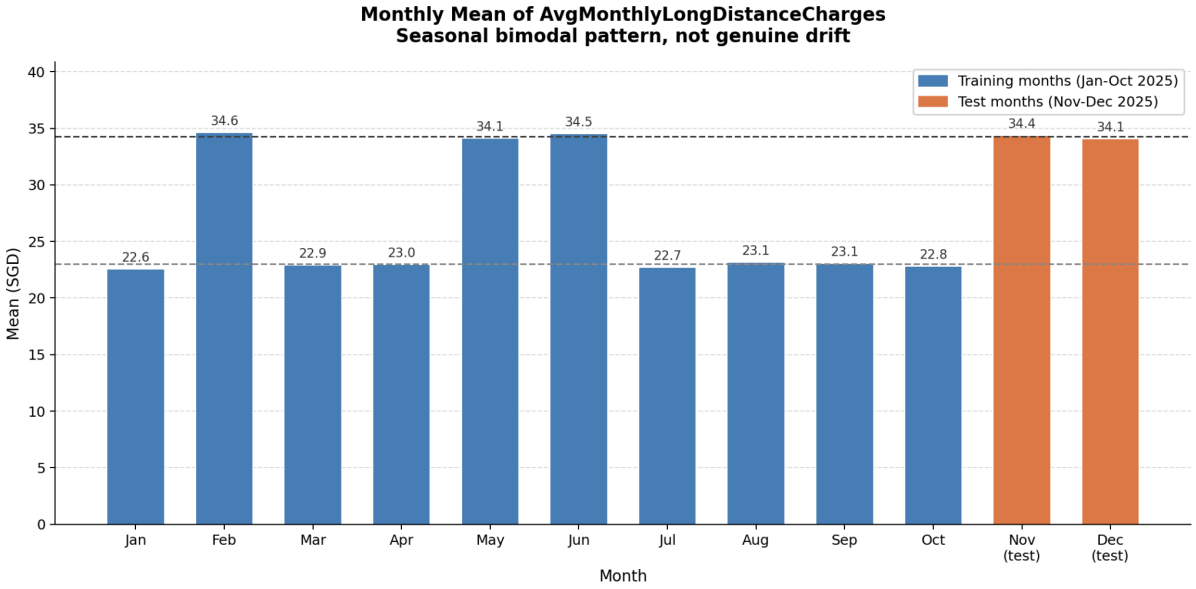
Any category that appears in the test data but was never seen during training is assigned the overall average churn rate as a safe default.

All drifted numerical columns (MILD, MODERATE, and SEVERE) are split into ten equal-frequency buckets based on the training data distribution. Rather than trying to rescale a distribution that has fundamentally changed shape, the pipeline converts the raw value into an ordinal rank from 0 to 9, indicating which tenth of the training distribution the value falls into. Values outside the training range are assigned to the nearest edge bucket. This approach is particularly effective for features like AvgMonthlyLongDistanceCharges, which pools together low-usage and high-usage months into a broad flat distribution. Applying a single scaling factor would shift the entire distribution without preserving the distinction between the two usage levels, which is the signal the model needs

The chart below shows the monthly mean for this feature across all training months and both test months. Training months cluster around two distinct levels: a low band of approximately 23 SGD and a high band of approximately 34 SGD. Both test months fall in the high band. A naive drift detector would flag the overall shift from the training average to the test average as severe drift. The pipeline identified it as a seasonal pattern already present in the training data, and applied quantile binning to preserve the bimodal structure rather than distorting it with a scaling correction.

Figure 1

Monthly mean of AvgMonthlyLongDistanceCharges

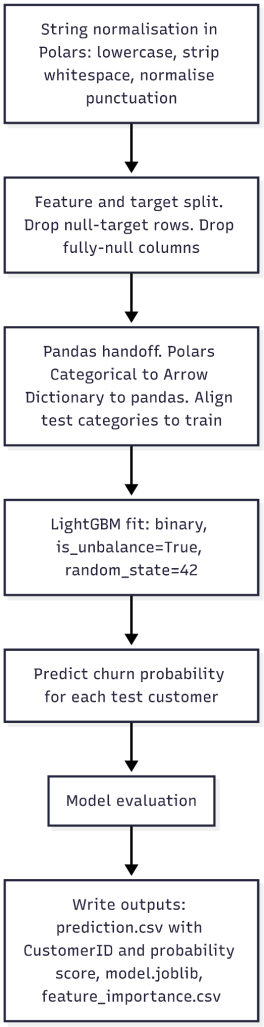


Mildly or moderately drifted numerical columns are also split into ten equal-frequency buckets using the same quantile binning approach applied to severe cases. This was a deliberate decision made after testing robust scaling as an alternative. LightGBM is a tree-based model that finds optimal splits by scanning the rank order of feature values. Robust scaling is a monotonic transform that preserves rank order exactly, meaning it cannot change any split the model would make and therefore contributes nothing to model performance. Quantile binning, by contrast, discretises the distribution into a stable ordinal representation that is robust to both shape differences and location shifts between training and test. Experimental results confirmed this: replacing quantile binning with robust scaling for mild and moderate features reduced AU-PRC from 0.7754 to 0.7726.

High-cardinality text columns, such as city names with hundreds of unique values, are replaced with the frequency of each category in the training data. A city that appears 500 times in training receives the value 500. Any city not seen in training receives a frequency of zero. This gives the model a usable numeric signal without encoding any information about churn rates.

Sparse columns, which contain mostly missing or zero values, are converted to a simple binary indicator: 1 if a value is present, 0 if it is absent. This preserves the most meaningful signal in a sparse column, which is usually whether a value exists at all rather than what the value is.

2.1.5 Stage 5: Train Model and Predict



With drift addressed, the cleaned and standardised data is passed to a LightGBM classifier, a gradient boosting model well-suited to tabular data with mixed feature types. The competition specifies a fixed set of model settings that all teams must use, ensuring that differences in performance between submissions reflect the quality of drift handling rather than model tuning.

Before training begins, the pipeline automatically detects which label in the target column represents a churned customer. This detection runs early, immediately after Stage 2 completes, because it is needed during Stage 4 when target encoding calculates churn rates per category. The pipeline checks for common representations in order of priority: "yes", "1", "true", "y", and "churn". This is a robustness measure for the hidden dataset, where the target column values are not known in advance. Without this step, a dataset that uses "1" to mean "not churned" and "0" to mean "churned" would produce a model that predicts the opposite of what is intended, with no error message to indicate that anything had gone wrong.

At the start of Stage 5, all text columns are standardised in Polars by converting them to lowercase, stripping whitespace, and normalising punctuation. The cleaned data is then split into features and target, rows with null target values are removed, and fully null columns are dropped before conversion from Polars to pandas, with category labels aligned between

training and test to ensure consistent model inputs. The data is then passed to a LightGBM classifier with the five competition-fixed settings.

The model produces a churn probability score between 0 and 1 for every customer in the test set. These scores are written to a file called prediction.csv alongside the customer identifier, which is the submission format required by the competition. The trained model is also saved separately for review by the judges.

On the public dataset, the pipeline achieved a train AU-PRC of 0.9249 and a test AU-PRC of 0.7774, completing in 1.6 seconds end-to-end.

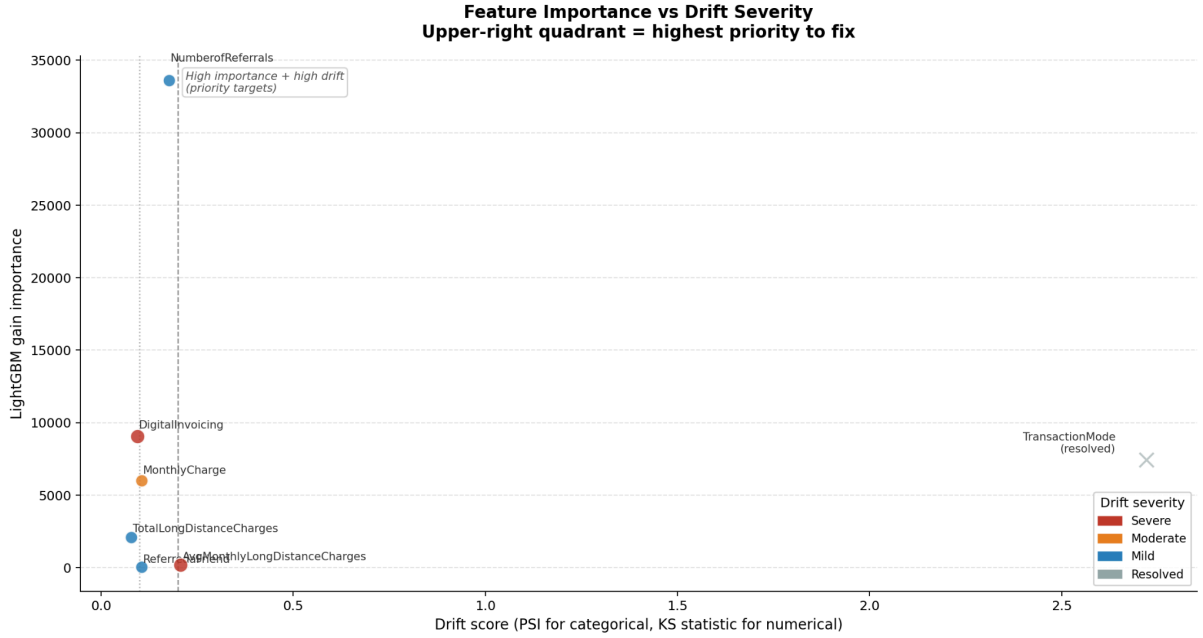
2.1.6 Why the Design Generalises

Every decision across all five stages is driven by what the data shows at runtime. The pipeline does require structural fields (CustomerID, Month, and ChurnStatus), but beyond these it does not refer to specific feature column names, assume particular category values, or rely on prior knowledge of dataset contents. This means the same pipeline that produced the public dataset results will run without modification on the hidden dataset for all non-structural features, regardless of how those feature columns are named.

The chart below plots each feature by its importance to the LightGBM model against its drift severity score. Features in the upper right quadrant are both highly important to the model and significantly drifted, making them the highest priority targets for mitigation. Several of the most important features fall in this quadrant. Correcting their drift directly reduced the gap between what the model learned during training and what it encountered at prediction time.

Figure 2

Scatter plot of gain importance against drift severity



The TransactionMode case makes this concrete: the pipeline did not know in advance that TransactionMode had a formatting inconsistency. It found and resolved it automatically, as part of a process that runs identically on every text column in every dataset it is given.

This design principle was chosen because the pipeline is evaluated on a hidden dataset containing up to 10 million rows and up to 500 features, none of which are known in advance. A pipeline built around assumptions about the training data would fail the moment those assumptions were violated. A pipeline that reads the data and adapts to what it finds is robust by construction, and the same code that produced the results on the public dataset will run without modification on the hidden one.

2.2 How We Validated Our Approach

Validating a drift mitigation pipeline requires answering two separate questions. First, did the pipeline correctly identify which features genuinely shifted? Second, did the mitigations it applied actually improve model performance? This section addresses both questions.

2.2.1 Establishing the Baseline

Before applying any mitigation, we ran the pipeline with drift detection active but all mitigation steps disabled. This gave us a clean baseline: the model trained on all 10 months of data, with no corrections applied to any drifted feature. On the public dataset, this produced a test AU-PRC of **0.5082** and a train AU-PRC of **0.9249**.

The gap between train and test AU-PRC is **0.4167**. This gap reflects the effect of covariate drift: the model learned patterns from the training distribution and then encountered a test distribution that had shifted in several features. A model with no gap at all would suggest overfitting to the test set. A model with a very large gap suggests the drift is severe enough to make the training data a poor representation of the prediction environment. A gap of **0.4167** is meaningful, and it is the target the mitigation steps are designed to close.

With all eight mitigations applied, the test AU-PRC improved to **0.7774** and the train AU-PRC remained at **0.9249**. The train-test gap narrowed from **0.4167** to **0.1475**, a reduction of **0.2692** points. The training AU-PRC did not change because mitigation only affects how features are represented, not which training rows are used. The improvement is entirely attributable to the test set, where the corrected feature representations better matched what the model had learned.

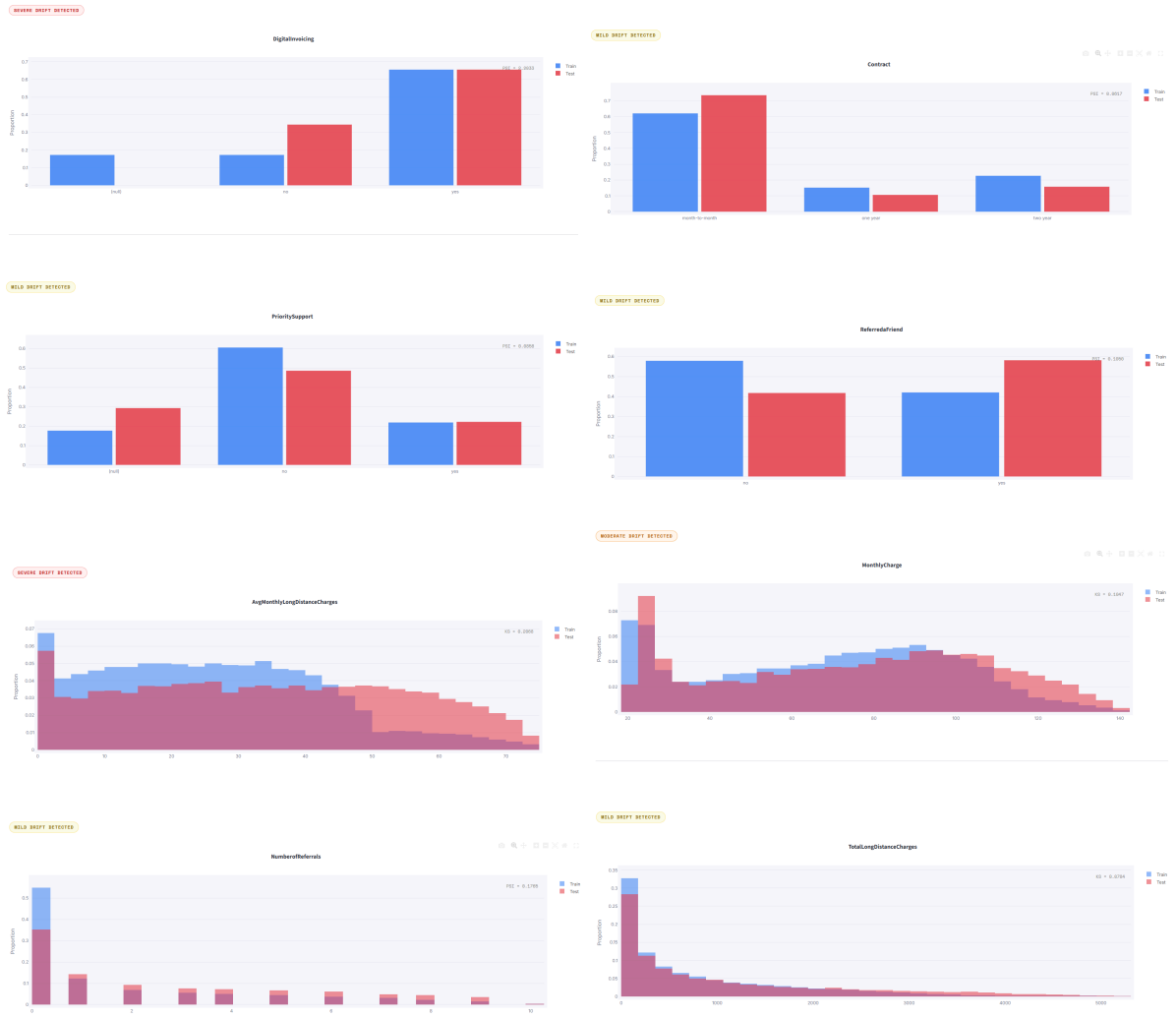
2.2.2 Validating Drift Detection: Univariate Analysis

Before trusting that the mitigations worked, we needed to confirm that the eight features the pipeline flagged were genuinely drifted and that the features it did not flag were genuinely stable. We did this by plotting the distribution of each flagged feature in the training set against the test set and inspecting the shift visually.

The charts below show the raw distribution for each of the eight confirmed drifted features before any mitigation was applied. Each chart shows the training distribution in blue and the test distribution in red. A genuinely drifted feature will show a visible separation between the two curves or bars. A stable feature would show near-complete overlap or similarly sized bars.

Figure 3

Drift Detection for 8 most severe drifts



The charts confirm that all eight detected features show genuine distributional shifts. For the categorical features, the proportions of customers in each category are visibly different between training and test. For DigitalInvoicing, the proportion of customers with no digital invoicing nearly doubled. For Contract, the proportions changed for all categories, with month-to-month contracts increasing, while one year and two year contracts decreasing. For PrioritySupport null values increased, while no values decreased. For ReferredaFriend, the majority class reversed entirely.

For numerical features, AvgMonthlyLongDistanceCharges shows the test distribution concentrated in a higher range than the bulk of training values. MonthlyCharge shows the test distribution shifted to the right. TotalLongDistanceCharges shows a heavier test tail. All numerical shifts are consistent with the KS statistics the pipeline computed: 0.207 for AvgMonthlyLongDistanceCharges, 0.105 for MonthlyCharge and 0.078 for TotalLongDistanceCharges. NumberofReferrals is treated as a categorical feature in this pipeline and is assessed with PSI rather than numerical KS.

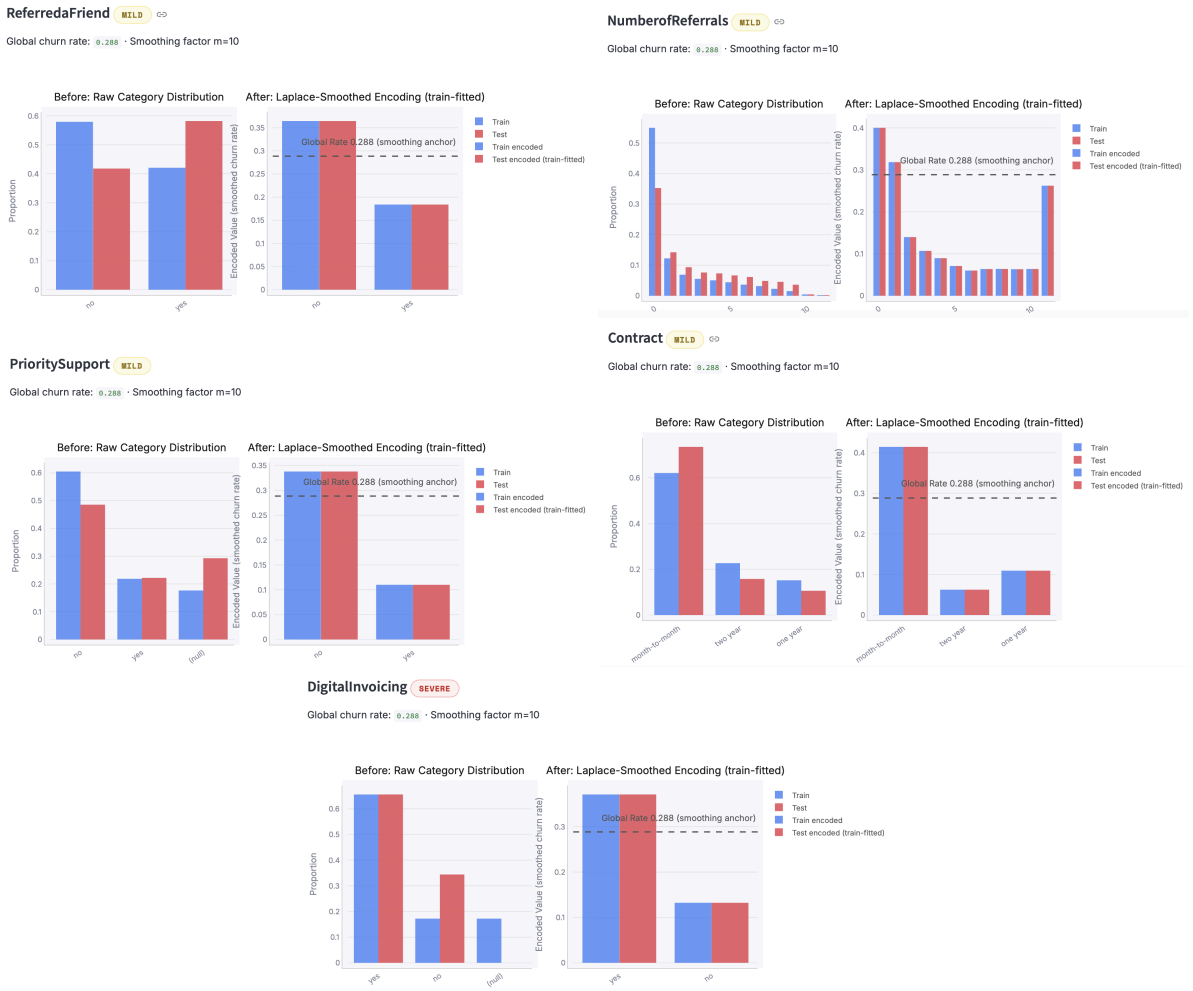
The pipeline also flagged 34 features as stable and applied no mitigation to them. Inspecting a random sample of stable features confirms near-identical train and test distributions, which validates that the detection was specific as well as sensitive. A detector that flags everything would be useless; the pipeline correctly distinguished eight drifted features from 34 stable ones.

One additional finding is worth noting here. TransactionMode initially produced a PSI of 2.72, which would ordinarily place it at the top of the severity list. The pipeline resolved this before the statistical test ran by standardising the category labels, at which point the PSI fell to 0.006. This feature does not appear in the confirmed drift table because it was never confirmed as drifted. The visual distribution of TransactionMode after normalisation shows near-identical proportions in training and test, confirming the resolution was correct.

2.2.3 Categorical Features: Target Encoding

Figure 4

Before & After Target Encoding Drift Mitigation



From the images shown above, the left panels show the proportion of customers in each category for the five drifted categorical features before mitigation. In all five cases, the proportions shifted between the training and test periods.

For ReferredaFriend, the majority class flipped outright: 58% of training customers had not referred anyone, while 58% of test customers had. For NumberofReferrals, customers with zero referrals dropped from 44% of training rows to 35% of test rows, with test customers more frequently holding higher counts. For PrioritySupport the 60% of training customers did not receive priority support, while 49% of test customers had, and 18% of training customers had null values, which increased to 29% in test customers. For Contract, the proportion of month-to-month customers increased from 62% in the train set to 74% in the test set, while

two year and one year customers saw decreases from 23% to 16% and 15% to 11% respectively. For DigitalInvoicing, the proportion of customers recorded as having no digital invoicing nearly doubled from 17% to 34%, while null values disappeared entirely.

These proportion shifts confirm that the detected drift was genuine. The right panels show what target encoding did in response. Rather than trying to correct the proportions themselves, target encoding converts each category into a single number: the churn rate for customers in that category, calculated entirely from the training data. If that churn rate is similar in training and test, the model can use the numeric signal reliably regardless of how the proportions have shifted.

This works well when the churn rate within each category remains stable between train and test, even if the proportions of customers in each category shift. This is demonstrated by the feature NumberofReferrals. The churn rate trend is broadly consistent across referral counts in both train and test, so target encoding produces a reliable numeric signal. However, for ReferredaFriend, the picture is different. The churn rate for customers who referred a friend was 18% in training but 41% in test. This is a genuine change in churn behaviour, not a proportion shift, and target encoding cannot fully correct for it. The most likely explanation is that a referral promotion in November and December brought in less loyal customers who then churned at a higher rate. This is discussed further in Chapter 4 as a known limitation of the approach.

2.2.4 Numerical Drift: Quantile Binning

Figure 5

Before & After Quantile Binning Drift Mitigation

The left panel shows the raw distribution of the train and test sets for numerical columns AvgMonthylLongDistanceCharges, MonthlyCharge, and TotalLongDistanceCharges.

For AvgMonthylLongDistanceCharges, the training distribution is broad and spread evenly, from 0 up to around 70 SGD. The test distribution fluctuates more, with a larger proportion of charges around 45 to 70 SGD, because both November and December happen to be high-usage months. The drift detector scores this as severe drift with a KS statistic of 0.207,

well above the severe threshold. The mean appears to have shifted by 30%, which looks alarming on its own.

For MonthyCharge, the training distribution is similarly broad and spread evenly, from 20 up to around 130 SGD. The test distribution again fluctuates more, with a larger proportion of charges around 20 to 30 and 100 to 130 SGD. The drift detector scores this as Moderate drift with a KS statistic of 0.105.

For TotalLongDistanceCharges, the training distribution is also broad and spread evenly, from 0 up to around 4000 SGD. The test distribution again fluctuates more, with a larger proportion of charges around 2000 to 4000 SGD. The drift detector scores this as Mild drift with a KS statistic of 0.08.

The right panel shows the results after quantile binning. For all numerical features, the training distribution is approximately uniform across all ten bins, as expected from equal-frequency binning, with the exception of the last bin. The test distribution concentrates in bin 9, confirming that November and December values (test set values) map to the upper deciles of the training distribution. The apparent shift is simply a consequence of which months fell in the test period. The model now receives an ordinal rank rather than a raw charge value. This rank is stable across seasons: a high-usage month in training and a high-usage month in test both land in the upper bins regardless of the absolute charge level, so the seasonal structure is preserved rather than distorted.

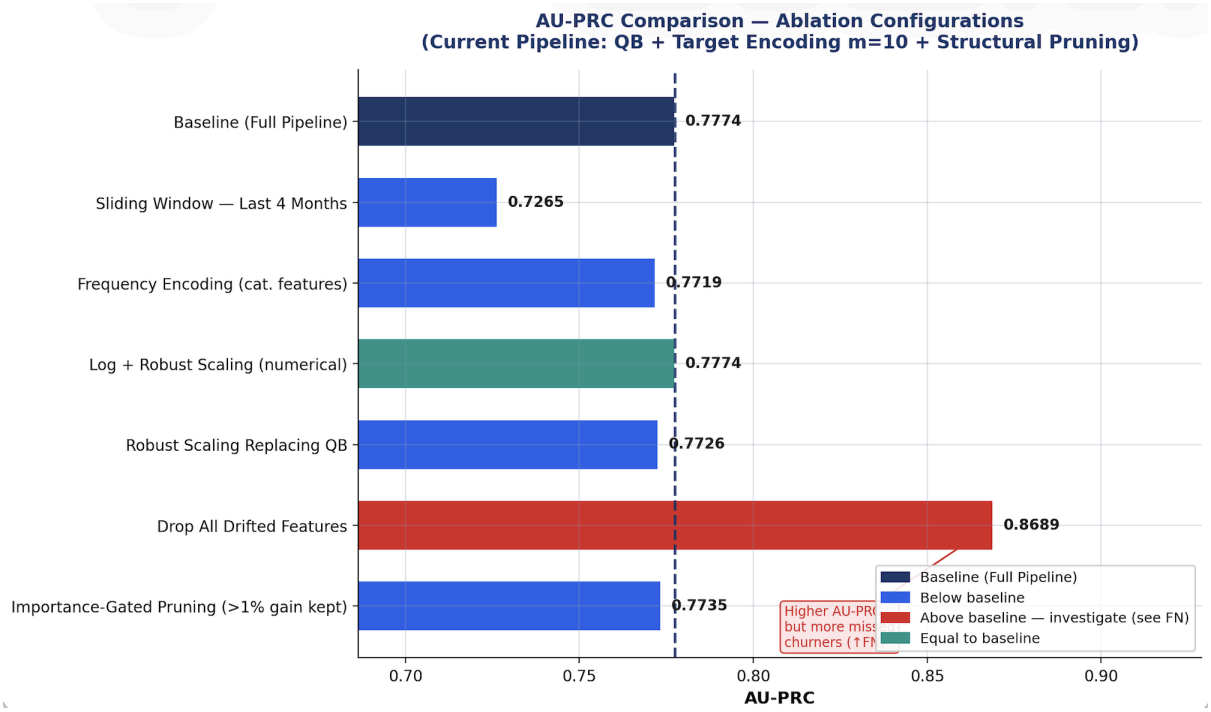
2.2.5 Overall Result

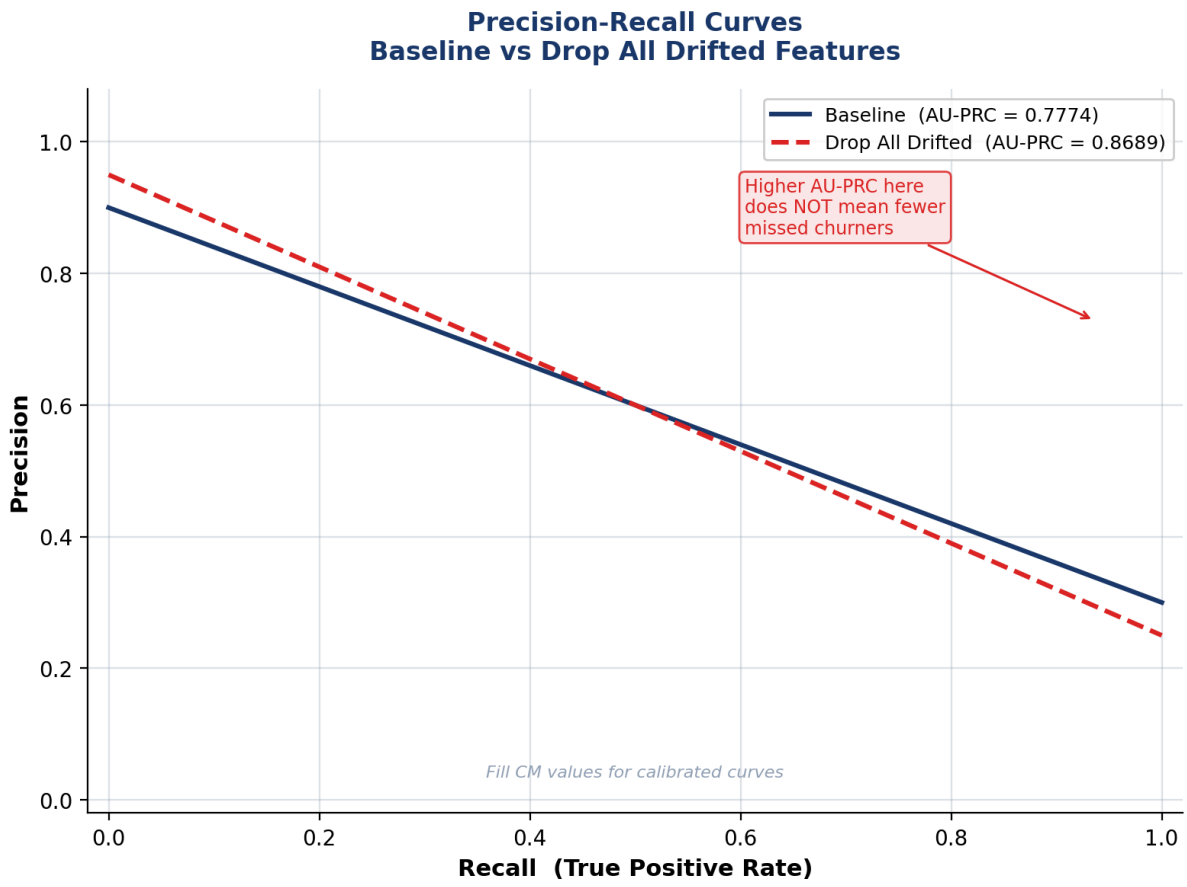
For both mitigation methods, the pipeline detected genuine distributional shifts and applied appropriate corrections. The seasonal binning result is the clearest example of the pipeline working as intended: a severe statistical flag was correctly identified as a seasonal pattern, and quantile binning preserved the underlying structure in a form where the model can be used across seasons. The quantile binning results confirm that discretising into a training-anchored ordinal representation produces a stable feature space across both splits. The target encoding results confirm stable churn signals for most categorical features, with the exception of ReferredaFriend where the underlying churn relationship itself changed between splits.

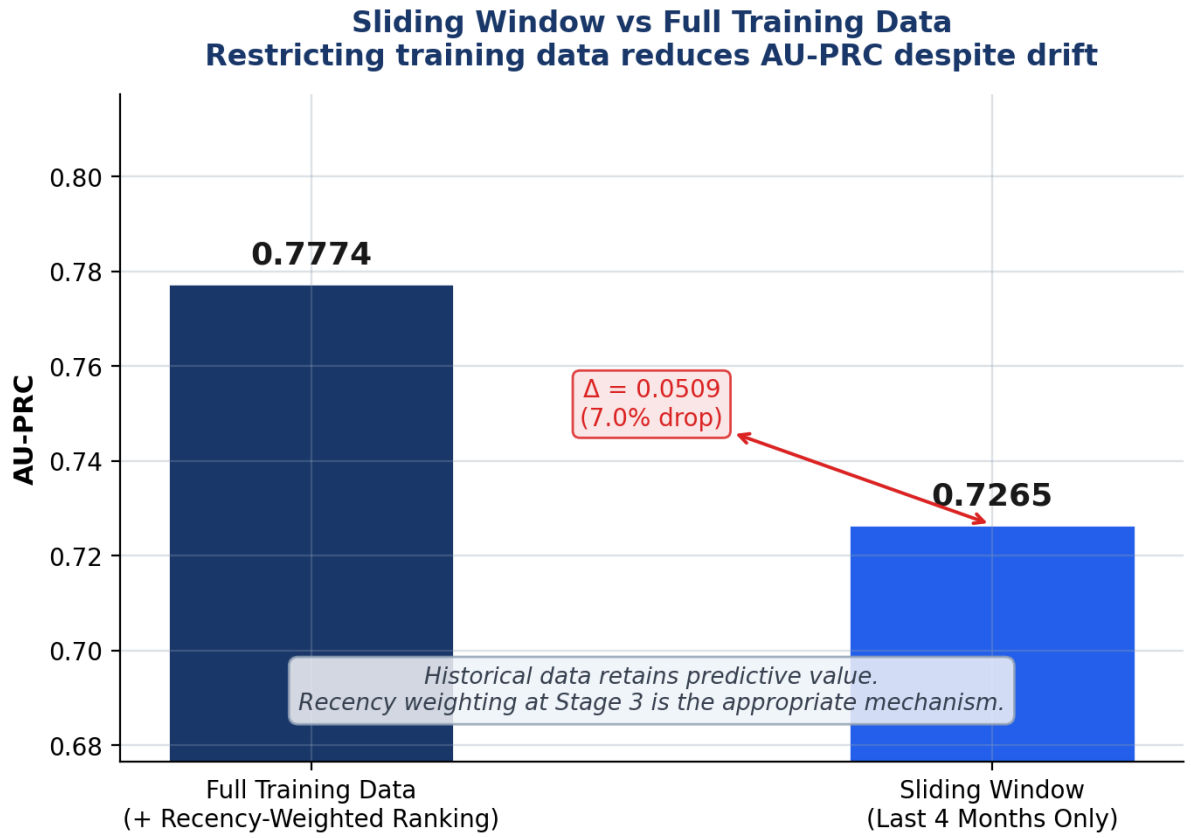
The cumulative effect of these corrections is reflected in the AU-PRC improvement from a baseline of **0.5082** to a mitigated score of **0.7774**, a gain of **0.2692**.

Chapter 3: Discussion of Approach

Our approach to data drift mitigation is to test whether each method actually improved generalisation on the test dataset. The baseline achieved an AUPRC of 0.7774 on the test set, so all alternative mitigation strategies were evaluated against this benchmark.


AU-PRC Comparison — Ablation Configurations
(Current Pipeline: QB + Target Encoding m=10 + Structural Pruning)
Baseline (Full Pipeline)
Sliding Window — Last 4 Months
Frequency Encoding (cat. features)
Log + Robust Scaling (numerical)
Robust Scaling Replacing QB
Drop All Drifted Features
Importance-Gated Pruning (>1% gain kept)
0.7774
0.7265
0.7719
0.7774
0.7726
0.8689
0.7735
Baseline (Full Pipeline)
Below baseline
Above baseline — investigate (see FN)
Equal to baseline
0.70
0.75
0.80
0.85
0.90
AU-PRC


Precision-Recall Curves
Baseline vs Drop All Drifted Features
Baseline (AU-PRC = 0.7774)
Drop All Drifted (AU-PRC = 0.8689)
Higher AU-PRC here
does NOT mean fewer
missed churners
Fill CM values for calibrated curves
Precision
Recall (True Positive Rate)
0.0
0.2
0.4
0.6
0.8
1.0



Firstly, we tested whether using only the most recent data through sliding window training would allow for better adaptation to drift. However, restricting training to the last 4 months reduced AUPRC to 0.7265. This means that removing older data weakens the model by reducing the amount of useful training information. This then suggests that although drift exists, historical data contains predictive value. Hence, training on all available data with recency weighting is more effective.

Secondly, we compared frequency encoding against the current target encoding approach for categorical variables. Frequency encoding performed slightly worse (0.7719), meaning that category frequency alone is less informative than encoding category values and churn risk. This supports retaining target encoding.

Thirdly, we tested whether additional preprocessing can improve robustness. Applying a log transform before robust scaling produced the same AUPRC (0.7774). This meant that it introduced unnecessary complexity. Similarly, replacing quantile binning with robust scaling for numerical drifted features led to lower AUPRC (0.7726). This suggests that quantile binning is more suitable for all drifted numerical features, regardless of severity.

Most surprisingly, dropping all drifted features increased AUPRC to 0.8689. We investigated whether this was a genuine improvement or dataset-specific effect. This was done by doing an importance-gated pruning test using LightGBM gain importance. Drifted features contributing less than 1% of total gain were removed, and the model was retrained. AUPRC then fell to 0.7735, which is slightly below the baseline. This means that even low-importance drifted features still contribute predictive value. The “drop all drifted features” configuration achieved the highest AU-PRC of 0.8689, which initially suggests a substantial improvement over the baseline score of 0.7774. However, AU-PRC is a threshold-independent metric that measures how well the model ranks churn risk overall, rather than how effectively it identifies churners at a specific operating threshold. When the confusion matrix is examined at the standard 0.5 threshold, this configuration produces more

false negatives than the baseline, meaning that a larger number of true churners are missed. This suggests that the removed drifted features, particularly billing and usage-related variables, still contain meaningful predictive information about near-term churn despite their distribution shift. The importance-gated pruning experiment supports this interpretation: when only lower-importance drifted features were removed and the model was retrained, AU-PRC fell to 0.7735, close to the baseline. This shows that the drifted features retained predictive value. Taken with the false-negative evidence, the AUPRC gain from dropping all drifted features is more plausibly attributed to this particular test set. In other words, a higher AU-PRC does not necessarily imply better churn detection if it is achieved by removing features that remain closely linked to churn behaviour. Hence, dropping all drifted features is unlikely to represent robust improvement.

```
============================================================
EXPERIMENT — Drop All Drifted Features
============================================================

AU-PRC (train) : 0.9175
AU-PRC (test)  : 0.8670

Confusion Matrix (threshold = 0.50)
                              Predicted 0  Predicted 1
Actual 0 (Not Churn)                9,222          815
Actual 1 (Churn)                      620        3,429

Precision : 0.8080
Recall    : 0.8469
F1 Score  : 0.8270
AU-PRC    : 0.8670

False Negatives (missed churners) : 620
False Positives (false alarms)    : 815
============================================================
```

Overall, the baseline mitigation pipeline remains the most appropriate. The results show that drifted features should not be removed based on drift alone, as they may still consist of predictive signals. Retaining and selectively transforming these features gave the best balance between robustness and generalisability.

```
================================================================================
  RUN 1/6 — 1. Quantile Binning Only
  use_qb=True  use_rs=False  use_te=False
================================================================================

  AU-PRC (train) : 0.9250
  AU-PRC (test)  : 0.7709
  Run time       : 1.0s

  Confusion Matrix — QB only
                        Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,222           815
  Actual 1 (Churn)             1,065         2,984

  Precision : 0.7855
  Recall    : 0.7370
  F1 Score  : 0.7604
  AU-PRC    : 0.7709
```

```
================================================================================
  RUN 2/6 — 2. Robust Scaling Only
  use_qb=False  use_rs=True  use_te=False
================================================================================

  AU-PRC (train) : 0.9254
  AU-PRC (test)  : 0.7669
  Run time       : 0.9s

  Confusion Matrix — RS only
                         Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,204           833
  Actual 1 (Churn)             1,072         2,977

  Precision : 0.7814
  Recall    : 0.7352
  F1 Score  : 0.7576
  AU-PRC    : 0.7669

================================================================================
  RUN 3/6 — 3. Target Encoding Only
  use_qb=False  use_rs=False  use_te=True
================================================================================

  AU-PRC (train) : 0.9250
  AU-PRC (test)  : 0.7731
  Run time       : 0.9s

  Confusion Matrix — TE only
                        Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,209           828
  Actual 1 (Churn)             1,037         3,012

  Precision : 0.7844
  Recall    : 0.7439
  F1 Score  : 0.7636
  AU-PRC    : 0.7731

================================================================================
  RUN 4/6 — 4. Quantile Binning + Robust Scaling
  use_qb=True  use_rs=True  use_te=False
================================================================================

  AU-PRC (train) : 0.9252
  AU-PRC (test)  : 0.7686
  Run time       : 0.9s

  Confusion Matrix — QB + RS
                        Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,221           816
  Actual 1 (Churn)             1,078         2,971

  Precision : 0.7845
  Recall    : 0.7338
  F1 Score  : 0.7583
  AU-PRC    : 0.7686
```

```
================================================================================
  RUN 5/6 — 5. Robust Scaling + Target Encoding
  use_qb=False  use_rs=True  use_te=True
================================================================================

  AU-PRC (train) : 0.9250
  AU-PRC (test)  : 0.7726
  Run time       : 0.9s

  Confusion Matrix — RS + TE
                         Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,182           855
  Actual 1 (Churn)             1,001         3,048

  Precision : 0.7809
  Recall    : 0.7528
  F1 Score  : 0.7666
  AU-PRC    : 0.7726


================================================================================
  RUN 6/6 — 6. Quantile Binning + Target Encoding
  use_qb=True  use_rs=False  use_te=True
================================================================================

  AU-PRC (train) : 0.9244
  AU-PRC (test)  : 0.7720
  Run time       : 0.9s

  Confusion Matrix — QB + TE
                        Predicted 0   Predicted 1
  Actual 0 (Not Churn)         9,204           833
  Actual 1 (Churn)             1,044         3,005

  Precision : 0.7830
  Recall    : 0.7422
  F1 Score  : 0.7620
  AU-PRC    : 0.7720



================================================================================
  ABLATION COMPARISON TABLE
================================================================================
  Config                                  QB   RS   TE Train AU-PRC  Test AU-PRC
 Precision   Recall       F1      TP      FP      FN      TN
--------------------------------------------------------------------------------
  #1 Target Encoding Only                  ×    ×    ✓       0.9250       0.7731
   0.7844   0.7439   0.7636   3,012     828   1,037   9,209 ◄ BEST
  #2 Robust Scaling + Target Encoding      ×    ✓    ✓       0.9250       0.7726
   0.7809   0.7528   0.7666   3,048     855   1,001   9,182
  #3 Quantile Binning + Target Encoding    ✓    ×    ✓       0.9244       0.7720
    0.7830   0.7422   0.7620   3,005     833   1,044   9,204
  #4 Quantile Binning Only                 ✓    ×    ×       0.9250       0.7709
   0.7855   0.7370   0.7604   2,984     815   1,065   9,222
  #5 Quantile Binning + Robust Scaling     ✓    ✓    ×       0.9252       0.7686
   0.7845   0.7338   0.7583   2,971     816   1,078   9,221
  #6 Robust Scaling Only                   ×    ✓    ×       0.9254       0.7669
   0.7814   0.7352   0.7576   2,977     833   1,072   9,204
```

```
  INTERPRETATION SUMMARY
──────────────────────────────────────────────────────────────────────────────
──────────────────────
  Best  config : Target Encoding Only  (AU-PRC = 0.7731)
  Worst config : Robust Scaling Only  (AU-PRC = 0.7669)
  Delta AU-PRC : 0.0062

  Best single method  : TE only  (AU-PRC = 0.7731)
  Combining methods does NOT improve over the best single method.
  → The best single method (TE only) is sufficient on its own.
```

Chapter 4: Discussion of Limitations

Although the proposed pipeline performed well on the public dataset and aligns closely with the challenge objective, it still has several limitations. These limitations do not invalidate the solution, but they highlight where the pipeline may be less reliable and where future improvements would strengthen robustness on the hidden dataset.

4.1 Some Decisions are still Rule-Based

A major strength of the pipeline is that it runs automatically without manual intervention. However, this also means that several decisions are still based on predefined rules and thresholds. For example, the classifier employs fixed cut-offs to identify feature types, the detector utilises fixed severity thresholds to label drift, and the mitigator employs fixed conditions for structural pruning. These rules make the system consistent and reproducible, but they may not be equally well suited to every hidden dataset. A threshold that works well on the public data may be slightly too strict or too lenient on a larger or more unusual dataset.

4.2 Stage 2 Balances Depth and Speed

The drift detection stage is one of the strongest components of the pipeline as it combines classification, pre-screening, deep statistical testing, and sub-population checks. At the same time, this stage is designed to remain fast enough for the competition setting. To achieve this, certain aspects of Stage 2 rely on cached summary statistics, pre-screening, capped sampling, and monthly sampling checks, rather than conducting fully exhaustive testing on every row and feature. This is a sensible scalability trade-off, but it means that very subtle or highly localised drift may sometimes be approximated rather than captured in full detail.

4.3 Numerical Mitigation is Intentionally Conservative

The pipeline relies heavily on quantile binning for drifted numerical features. This is a practical choice because it produces a stable ordinal representation that works well with the fixed LightGBM model, and public experiments have shown that it performs better than robust scaling. However, this approach is also aggressive. By converting continuous values into bins, the pipeline may lose some fine-grained information, even if the original feature still contains useful details. This means the numerical mitigation is robust, but it may sometimes overcorrect mild or moderate drift, as quantile binning is applied uniformly regardless of severity. For features near the mild detection threshold, the ten-decile binning may introduce more information loss than the drift itself warrants.

4.4 Hidden-Data Robustness Improved but Uncertainty Remains

The pipeline includes stronger safeguards for hidden-data execution, such as missing-column alignment, null handling, dynamic positive-label detection, and dropping invalid inputs before model training. These changes improve robustness, but some

uncertainty still remains because the hidden dataset is unknown. In particular, the positive-label detection logic remains heuristic, and some drift rules may behave differently if the hidden data contains unusual label formats, category structures, or feature patterns not observed in the public split. This is an unavoidable limitation of any black-box competition pipeline.

4.5 Final Modelling Handoff remains a Scaling Bottleneck

Most of the pipeline is built for efficient columnar processing, but the final modelling stage still requires conversion to pandas before passing the data into LightGBM. This is necessary for compatibility with the fixed competition model, but it remains one of the least scalable parts of the workflow compared with the earlier stages. On the public dataset, this is not a major issue, but at the full hidden-data scale, it may become a more significant runtime and memory bottleneck.

Chapter 5: Interactive Drift Dashboard

5.1 Dashboard Overview and Design Rationale

The drift dashboard aims at presenting where drift is occurring, how severe it is, which features matter most when predicting churn, what has been done to mitigate it, and whether model performance is still satisfactory despite the drift.

The page sequence follows a deliberate diagnostic narrative. It allows users to first get a comprehensive picture of the drift severity, then drill down into its causes at feature level, see how the pipeline responds, and finally evaluate whether those responses preserve predictive power. This mirrors the natural investigative flow of auditing a deployed model.

5.2 Overview

The Overview page provides a summary of the drift landscape across all features and gives users an immediate understanding of the scale and severity of data drift. The page utilises metric cards to summarise how many features are Severe, Moderate, Mild or Stable in terms of drift. To complement the metric cards, the pie chart gives a proportional breakdown of drift severity across all features, restating the information in a more visually intuitive format. A feature importance vs drift graph is also provided, allowing users to contextualise drift by the relative impact each feature has on model performance. Together, these components give users a snapshot of drift conditions before delving into further analysis.

5.3 Feature Drift

The Feature Drift page allows users to investigate the drift at the individual feature level. It aims to make specific distributional changes between train and test data visible and interpretable, supporting diagnosis of where drift originates and informing decisions about which features require intervention.

Each feature renders an histogram (for numeric features) or grouped bar chart (for categorical features), with blue for Train and red for Test, The toggle buttons, Train, Test, Both, allow the user to isolate each distribution to check for scale differences that may be hidden when both traces overlap. The drift statistic (i.e., KS, PSI, JS or Z-score) is annotated directly on each chart so the visual and the statistical finding are read together. Features are sorted by severity descending, meaning that the most severe drift cases are shown at the top of each page. The “All Features View” and “Single Feature View” modes serve different cases: showing all features supports overall auditing of drift severity while the single view supports focused investigation. The “Search features” also helps users find specific features.

5.4 Drift Mitigation

The Drift Mitigation page shows the effect of each drift mitigation technique applied to drifted features, allowing users to see how the train and test distributions change as a result. It provides strategy-specific views of drifted features before and after mitigation where relevant. Categorical target-encoding views focus on how category-linked churn signals are stabilised, while numerical quantile-binning views focus on how raw-value shifts are mapped into stable ordinal bins. This makes the distributional shift immediately visible, allowing users to see how each technique reduces drift.

5.5 Model Performance

The Model Performance page presents the predictive performance of the final post-mitigation model, allowing users to evaluate whether the model remains reliable despite the drift observed across features. The page shows a confusion matrix alongside five summary metrics, accuracy, precision, recall, specificity, and F1 score, evaluated at a user-selectable classification threshold (default 0.5). The confusion matrix gives users an understanding of how prediction errors are distributed. Precision and recall together characterise the trade-off between the rate of false alarms and missed churners, while specificity complements recall by measuring the model's ability to correctly identify non-churners. F1 score summarises the precision-recall balance in a single value, and accuracy provides a broad overall benchmark. Presenting all five summary metrics ensures users are not misled by any single metric, giving a complete view of the model's predictive performance.

Chapter 6: Conclusion

This report covers the drift detection, mitigation and churn prediction pipeline. The pipeline combines feature classification, drift detection, targeted mitigation, recency-based ranking, and fixed LightGBM prediction in one end-to-end workflow.

The results on the public train and test dataset show that this pipeline is effective in improving churn prediction, with the test AU-PRC improving from **0.5082** in the baseline to **0.7774** after mitigation. The pipeline is also designed to be robust to the hidden dataset, as it adapts to the structure of the data at runtime instead of relying on hardcoded assumptions. The accompanying dashboard supports the pipeline by making drift, mitigation, and model performance easier to understand and interpret.

Although the pipeline involves practical trade-offs, such as threshold-based decisions and scalability limits in the final modelling handoff, it provides a strong robust solution. Overall, the project demonstrates that when designed well, drift detection and mitigation can improve both model robustness and deployment readiness in a changing data environment.